Assumptions

1. An activity (stored in *activity_name*)represents a sport discipline (e.g., yoga, tennis, basketball), while a *Class* is a recurring course offering one activity taught by a specific instructor. This allows the same activity to be offered through multiple classes.
2. Instructor description. An instructor can be specialized in several activities, Thus, they can teach several classes.
3. Room assignment level. Rooms are assigned at the *Session* level, not at the *Class* level. The same class may take place in different rooms across its sessions, so each session stores the room in which it occurs.
4. Enrollment vs Attendance. *Enrollment* represents a member's registration to a class, while *Attendance* records whether a member actually attends a specific session of that class.
5. Session datetime format: `start_time` and `end_time` for sessions are stored as `DATETIME` values including both date and time; therefore, no separate date attribute is used.
6. Email uniqueness. Member emails are unique among members, and instructor emails are unique among instructors. This ensures that each email identifies at most one member and one instructor.
7. Standard class duration. Each activity has a predefined standard duration (e.g., basketball = 90 minutes), and all sessions of the same class respect this duration, meaning that `end_time - start_time` equals the standard duration defined for that activity.
8. A class needs at least one member enrolled to happen.

Part 1 - Requirements analysis

Main entities:

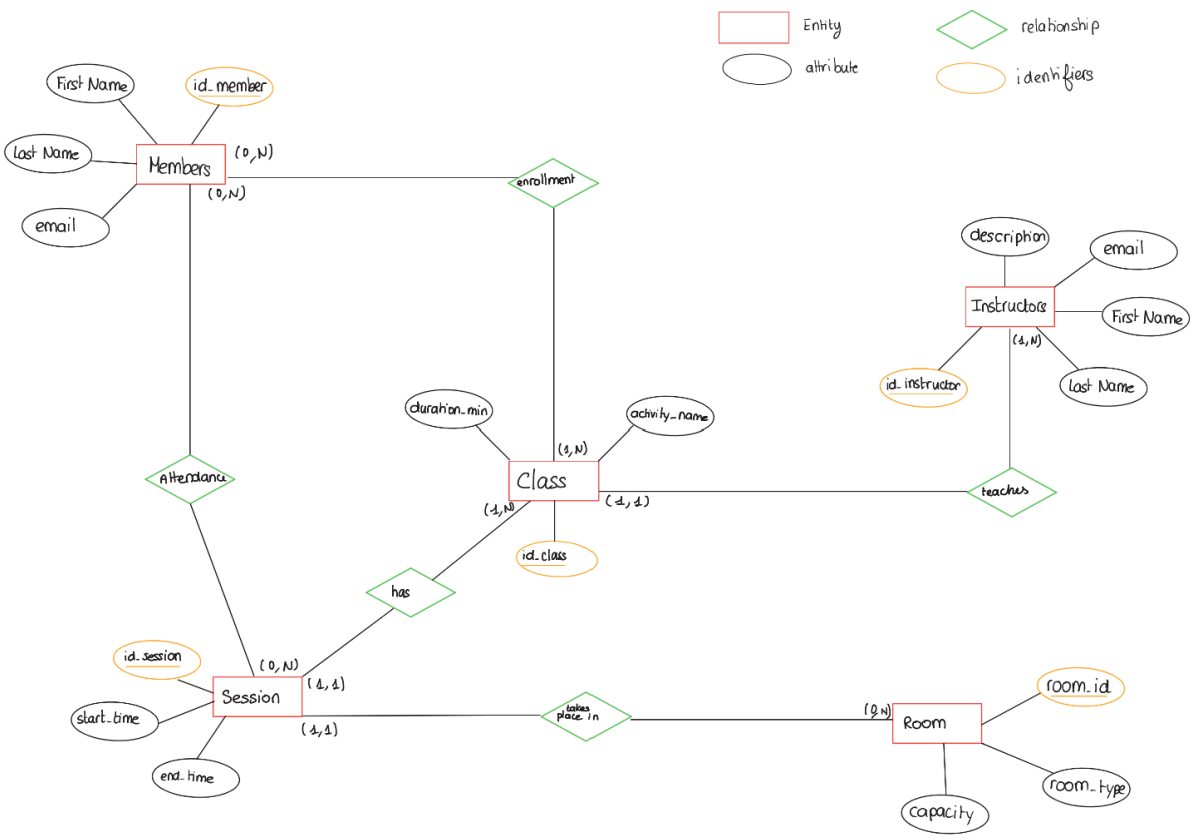
Entities	Description
Members	A person who joins the sports center
Instructors	A staff member of the center who teaches sports activities
Rooms	A place in the sports center where activities takes place
Sessions	A time slot dedicated to an class(date and time)
Classes	A recurring course offering an activity, taught by one instructor.

Entities	Attributes
Members	**id_member (PK), first_name, last_name, email.**
Instructors	**id_instructor (PK), first_name, last_name, email, description**
Rooms	**id_room (PK), room_type, capacity**
Sessions	**id_session (PK), id_class (FK), id_room (FK), start_time, end_time**
Classes	**id_class (PK), id_instructor (FK)**

Main Relationships and their cardinalities :

Relationships	Description
Members - Enrollment - Classes	A member can enroll to zero or several Classes(1:N) and Class welcomes several members (1:N)
Members - Attendance - Session	A member attends to zero or several sessions (1:N) and a Session can be attended by zero or several members (1:N)
Instructors - Teaches - Classes	An instructor can teach one or several activities (1:N) A class a taught by one instructor (1:1)
Session - Takes place in - Room	A Session takes place in a room (1:1) A room can host several session(1:N)
Class - Has - Session	A class has one or several sessions (1,N) A session belongs to exactly one class (1,1)

Part 4 - E-R Model:



Part 3 - Constraints

1. An instructor cannot teach two sessions with overlapping time. For any two sessions taught by the same instructor, their time slots must not overlap.
2. A member cannot enroll in a class if the number of enrolled members has already reached the capacity of the room assigned to the sessions of that class.
3. A member cannot be enrolled in two classes whose sessions overlap in time. At any given time slot, a member can only attend one session.
4. Two sessions can overlap in time only if they are held in different rooms. The same room cannot host two sessions with overlapping time intervals.
5. A class has a predetermined duration (e.g. basketball = 90 min). The difference between end_time and start_time must equal the standard duration defined for that class.
6. Email has to be unique for both members and instructors
7. End_time has to be greater than star_time
8. The number of participants in a session cannot exceed the room's capacity

Part 4- Relational schema

Members(id_member (PK), first_name, last_name, email)

Instructors(id_instructor (PK), first_name, last_name, email, description)

Rooms(id_room (PK), room_type, capacity)

Classes(id_class (PK), activity_name, duration_min, id_instructor (FK → Instructors))

Sessions(id_session (PK), id_class (FK → Classes), id_room (FK → Rooms), start_time, end_time)

Enrollment(id_member (FK → Members), (id_class FK → Classes))
PK: (id_member, id_class)

Attendance(id_member (FK → Members), id_session (FK → Sessions))
PK: (id_member, id_session)

- The primary key of Enrollment is the pair (`id_member`, `id_class`) to avoid duplicate enrollments of the same member in the same class.
- The primary key of Attendance is (`id_member`, `id_session`) so that a member can have at most one attendance record per session.

Part 5- SQL Create tables

```
CREATE TABLE Members (
id_member INT PRIMARY KEY,
first_name VARCHAR(50) NOT NULL,
last_name VARCHAR(50) NOT NULL,
email VARCHAR(100) UNIQUE
 );

CREATE TABLE Instructors(
id_instructor INT PRIMARY KEY,
first_name VARCHAR(50) NOT NULL,
last_name VARCHAR(50) NOT NULL,
email VARCHAR(100) UNIQUE,
description VARCHAR(100)
 );
```

```
CREATE TABLE Rooms(
id_room INT PRIMARY KEY,
room_type VARCHAR(50),
capacity INT NOT NULL CHECK (capacity > 0)
);

CREATE TABLE Classes(
id_class INT PRIMARY KEY,
activity_name VARCHAR(100) NOT NULL,
Duration_min INT NOT NULL,
id_instructor INT NOT NULL,
FOREIGN KEY (id_instructor) REFERENCES Instructors(id_instructor)
);


CREATE TABLE Sessions (
id_session INT PRIMARY KEY,
id_class INT NOT NULL,
id_room INT NOT NULL,
start_time DATETIME NOT NULL,
end_time DATETIME NOT NULL,
CHECK (end_time > start_time),
FOREIGN KEY (id_class) REFERENCES Classes(id_class),
FOREIGN KEY (id_room) REFERENCES Rooms(id_room)
 );



CREATE TABLE Enrollment (
id_member INT NOT NULL,
id_class INT NOT NULL,
PRIMARY KEY (id_member, id_class),
FOREIGN KEY (id_member) REFERENCES Members(id_member),
FOREIGN KEY (id_class) REFERENCES Classes(id_class)
);

CREATE TABLE Attendance (
id_member INT NOT NULL,
id_session INT NOT NULL,
PRIMARY KEY (id_member, id_session),
FOREIGN KEY (id_member) REFERENCES Members(id_member),
FOREIGN KEY (id_session) REFERENCES Sessions(id_session)
);
```

Part 6- SQL Queries

1 . List all members enrolled in a given class.

```
SELECT m.first_name, m.last_name
FROM Members as m
Inner join Enrollment as e
On m.id_member = e.id_member
Where e.id_class = 2;
```

2 . Show all sessions of a specific class including date and instructor.

```
Select s.id_session, s.start_time, s.end_time,  i.first_name, i.last_name
From Sessions as s
Inner Join Classes as c
On c.id_class=s.id_class
Inner join Instructors as i
On i.id_instructor=c.id_instructor
Where c.id_class = 2;
```

3. List the number of members enrolled in each class.

```
Select e.id_class, count(e.id_member) as total_enrolled
From Enrollment as e
Group by (id_class);
```

4. Find instructors who teach more than one class.

```
Select i.id_instructor, i.first_name, i.last_name, COUNT(c.id_class) AS number_of_classes
From Instructors as i
Inner join Classes as c
On i.id_instructor = c.id_instructor
Group by i.id_instructor, i.first_name, i.last_name
Having   COUNT(c.id_class) > 1 ;
```

5. List members who never attended any session.

```
Select m.id_member, m.first_name, m.last_name
From Members as m
Left join Attendance as a
On a.id_member = m.id_member
Where a.id_session IS NULL ;
```

6. Show the total attendance for each session

```
Select s.id_session, s.start_time, count(a.id_member) AS total_attendance
From Sessions as s
Left join Attendance as a
On  s.id_session = a.id_session
Group by  s.id_session, s.start_time;
```

Bonus Question:

```
Select c.activity_name, count(a.id_member) as total_attendance, i.first_name, i.last_name
From Classes as c

Inner join Sessions as s
On c.id_class = s.id_class

Inner join Attendance as a
On a.id_session = s.id_session

Inner join Instructors as i
On i.id_instructor=c.id_instructor

Group by c.id_class, c.activity_name,  i.first_name, i.last_name
Having count(a.id_member) > 10
Order by count(a.id_member) desc;
```

Contributions:

Bonnie-Kézia OWONO was responsible for the requirements analysis, the ER model, some of the constraints, and the SQL queries

Anna LEBRUN handled the relational schema, SQL table definitions, and some of the constraints